EventSync: Architecting a Multi-Client Event Scheduling and Resource Management System

Introduction and Problem Space Definition

The modernization of venue management and event scheduling requires transitioning from disjointed manual tracking methods and rudimentary spreadsheets to cohesive, intelligent systems capable of multi-dimensional constraint solving. The EventSync problem statement outlines a scenario that is highly representative of large-scale event management enterprises: the coordination of multiple geographic venues, fluctuating hall capacities, and a strictly constrained pool of shared operational resources, including projectors, audio systems, and human support staff[1]. The primary operational bottleneck in such high-stakes environments arises from overlapping temporal scheduling requests, last-minute client cancellations, and the manual reconciliation of shared equipment[1]. These manual processes frequently result in catastrophic double bookings, resource exhaustion during critical event phases, or the underutilization of premium venue space[1].

To secure top marks in the Mini Project ESE Hackathon 2026, the proposed web-based application must transcend basic data entry. It must seamlessly fulfill the foundational requirements—recording detailed client event requests, enforcing strict time operating restrictions, and maintaining absolute transactional integrity during complex booking modifications—while simultaneously implementing robust, mathematically sound conflict detection algorithms[1]. Furthermore, the hackathon rubric places a significant emphasis on an "Innovation Challenge," encouraging the development of high-value features that address genuine real-world venue management needs[1]. This report provides an exhaustive architectural blueprint, algorithmic strategy, and feature specification for the EventSync application. It details the optimal temporal data structures for overlap detection, the database constraints necessary for strict concurrency control, and a comprehensive suite of advanced innovative features designed to distinguish the solution within the competitive evaluation framework[2].

Architectural Strategy and Technology Stack Selection

The technological foundation of the EventSync platform must be chosen not merely for ease of use, but for its capability to handle the specific data integrity and temporal querying challenges inherent in resource scheduling. While the hackathon guidelines permit any suitable stack—including React, Angular, Node.js, Python, Java, MySQL, PostgreSQL, SQLite, or MongoDB—the nature of temporal overlap detection dictates a highly specific optimal path[1]. The analysis indicates that a decoupled architecture consisting of a React-based frontend and a Node.js/Express backend provides the necessary flexibility for rapid prototyping within the 10-12 hour development window[1]. However, the most critical architectural decision lies in the

database selection. While SQLite offers portability and simplicity, it fundamentally relies on a single-writer lock at the database level[5]. In a multi-client web architecture where concurrent booking requests are expected, SQLite serializes writers, creating a severe throughput bottleneck and placing the burden of temporal conflict resolution entirely on the application code[5]. MySQL introduces row-level locking via InnoDB, but historically struggles with enforcing complex temporal exclusion constraints without the use of cumbersome application-level triggers[6].

Therefore, PostgreSQL emerges as the strictly superior choice for the EventSync application[6]. PostgreSQL provides native support for range types, such as timestamp ranges (tsrange), and features the Generalized Search Tree (GiST) index[8]. This allows the database itself to enforce complex exclusion constraints, physically preventing two overlapping bookings for the same hall from ever being written to the disk, regardless of application-level race conditions[8]. Selecting PostgreSQL addresses the "reliable booking changes" requirement at the lowest possible architectural level, demonstrating a profound understanding of enterprise-grade data integrity to the hackathon judges[1].

Database Schema and Temporal Data Modeling

The foundation of the EventSync system relies on a highly normalized relational database architecture that accurately models the physical and temporal constraints of the venues.

Initial Dataset and Capacity Constraints

The system must initialize with the specific hierarchical relationship between venues, their constituent halls, and their localized shared resource pools[1]. The structural hierarchy is defined by the initial dataset provided in the problem constraints.

Venue	Halls (Seating Capacity)	Shared Resources (Total Pool per Venue)
Venue A	Hall A1 (60), Hall A2 (150)	2 Projectors, 2 Audio Systems, 5 Staff
Venue B	Hall B1 (100), Hall B2 (200)	2 Projectors, 1 Audio System, 4 Staff
Venue C	Hall C1 (80), Hall C2 (250)	3 Projectors, 2 Audio Systems, 6 Staff

To represent this programmatically while avoiding hardcoded values that would inhibit scalability, the database schema must strictly separate static infrastructural resources from temporal allocations[1]. The architecture necessitates the creation of several core relational entities. The Venue table stores the venue identifier, name, and operating boundaries,

specifically enforcing the 8:00 AM to 10:00 PM operational window[1]. The Hall table establishes a foreign key relationship to the venue and stores the maximum seating capacity[1]. The VenueResource table acts as a ledger for the shared equipment and personnel, mapping resource types (projectors, audio systems, staff) to their respective venues along with the total available quantity[1].

The transactional core of the application resides in the EventBooking table, which records the client identifier, hall assignment, and the critical temporal markers: the requested event start and end times, as well as the calculated reserved start and end times which include the mandatory operational buffers[1]. Finally, a BookingResourceAllocation junction table tracks the exact quantity of each shared resource consumed by a specific booking during its reserved temporal interval.

Temporal Boundaries and Half-Open Intervals

A critical functional constraint dictates that operating hours are strictly 8:00 AM to 10:00 PM, and every single booking requires a mandatory 30-minute buffer before the event start time and after the event end time for essential setup and cleanup operations[1]. Therefore, a client request for an event running from 10:00 AM to 1:00 PM mathematically translates to a continuous hardware, personnel, and spatial reservation spanning from 9:30 AM to 1:30 PM. The entirety of this expanded interval, not just the core event time, must fall completely within the 8:00 AM to 10:00 PM operational window[1].

To prevent boundary condition errors and overlapping logic failures, the system must model all time reservations using half-open intervals, mathematically represented as $[start, end)$ [7]. In standard SQL implementations, utilizing the BETWEEN operator creates a closed interval $[start, end]$, meaning both the upper and lower boundaries are inclusive[7]. If intervals are modeled as closed ranges, a reservation ending at exactly 10:00 AM and a subsequent reservation beginning at exactly 10:00 AM would incorrectly share a microsecond of overlap at the boundary, falsely triggering a resource exhaustion or double-booking alert[7].

By standardizing all algorithms and database queries to utilize the half-open paradigm, the upper boundary is strictly exclusive[7]. A booking interval from 9:00 AM to 10:00 AM includes all time up to, but not including, the exact stroke of 10:00 AM. Consequently, evaluating an overlap between two intervals, A and B, simplifies to the boolean expression: $start_A < end_B \land start_B < end_A$ [9]. If Interval A is [9:00, 10:00) and Interval B is [10:00, 11:00), the condition $10:00 < 10:00$ evaluates to false, correctly confirming that back-to-back scheduling is perfectly valid and causes no temporal conflicts[12]. This strict adherence to half-open interval logic is non-negotiable for producing an enterprise-grade scheduling engine.

Concurrency Control and the Limitations of Standard ORMs

While the application layer will handle the preliminary business logic and conflict detection,

relying solely on application-memory algorithms introduces severe race conditions in any multi-client web architecture[14]. If two concurrent HTTP requests from different clients attempt to book Hall A1 for the exact same time slot, both application threads may query the database simultaneously. Both threads will read the hall as "available" before either has the opportunity to write the confirmed booking to the database, resulting in a catastrophic double-booking that violates the core hackathon constraints[1].

PostgreSQL Exclusion Constraints

To achieve absolute data integrity without sacrificing concurrency, the system must bypass the limitations of standard Object-Relational Mappers (ORMs) and implement database-level exclusion constraints[8]. As previously established, PostgreSQL allows the definition of an EXCLUDE USING gist constraint[8]. The database migration must execute the following SQL definition to guarantee spatial isolation:

The constraint operates by specifying that for any given row in the EventBooking table, if the HallID is identical to another row (evaluated using the = operator), their reserved timestamp ranges (tsrange) must not overlap (evaluated using the && overlap operator)[8]. Crucially, the syntax [) is explicitly passed to the tsrange function to enforce the half-open interval logic directly at the database engine level[8]. Furthermore, this constraint is scoped with a WHERE (Status = 'Confirmed') clause, ensuring that cancelled or pending requests do not artificially lock the hall[1].

This implementation provides an impenetrable safeguard. If the application server algorithms fail to catch a conflict, or if a race condition occurs, the PostgreSQL engine will physically reject the second transaction, throwing an IntegrityError[6]. The Node.js backend can gracefully catch this specific error and translate it into a polite, actionable user-facing message, perfectly satisfying the "decision visibility" requirement by explaining exactly why the request could not be accommodated[1].

Algorithmic Foundations: Overlap Detection and Resource Aggregation

The most computationally complex requirement of the EventSync application is verifying the availability of both distinct physical spaces (halls) and aggregated shared resources across continuously overlapping timeframes[1]. The architecture requires the synthesis of two distinct algorithmic paradigms to solve this efficiently: the Interval Tree for querying spatial availability and the Sweep Line algorithm for verifying aggregate resource limits.

Hall Selection and Tie-Breaking Logic

The problem statement establishes a strict hierarchical search methodology for hall allocation that the algorithm must execute sequentially:

1. Select the smallest-capacity hall that satisfies the expected attendee count.
2. Search the client's preferred venue first.
3. If the preferred venue has no feasible halls, or lacks the required shared resources during the requested interval, systematically search alternative venues.
4. Apply a consistent tie-breaker when multiple choices present equal viability[1].

When a scheduling payload arrives at the backend, the system first retrieves all halls globally where the seating capacity is greater than or equal to the requested attendee count[1]. This candidate list must then be deterministically sorted using a multi-key comparator. The primary sort key is the hall capacity in ascending order, strictly fulfilling the "smallest-capacity" mandate[1]. The secondary sort key is a boolean flag indicating whether the hall belongs to the client's preferred venue, sorted in descending order to prioritize true values. Finally, the tertiary sort key acts as the tie-breaker; the hall identifier is sorted lexicographically in ascending order[1]. This guarantees that if a venue possesses two identical 100-person halls, the algorithm will consistently attempt to book the first hall (e.g., Hall B1 before Hall B2) rather than behaving randomly, satisfying the tie-breaker constraint[1]. Once this strictly ordered candidate list is generated, each hall must undergo temporal conflict verification.

The Interval Tree: Spatial Availability

Checking if a specific hall is available requires evaluating the new requested time interval against all existing confirmed bookings for that hall. A naive brute-force approach would iterate through every booking in the database, resulting in an $O(N)$ time complexity per query, which rapidly degrades system performance as the venue history grows[11]. To optimize this, the system implements an Interval Tree[11].

An Interval Tree is a specialized, self-balancing binary search tree specifically engineered to store and query continuous time ranges efficiently[11]. In the EventSync implementation, the application maintains an in-memory Interval Tree of active bookings for each hall. Each node within the tree stores a booking interval $[low, high)$, and the Binary Search Tree is structurally ordered by the low value (the reserved start time)[11]. The critical augmentation that distinguishes an Interval Tree from a standard BST is that each node also stores a max_high value, representing the maximum end time present anywhere within its entire subtree[11].

When querying if a new event's required interval $[q_{low}, q_{high})$ conflicts with existing bookings, the search algorithm begins at the root node and traverses downward[11]. At any given node, it checks for a direct overlap. Then, it evaluates the left child. If the left child's max_high value is strictly less than q_{low}, the algorithm has mathematical proof that no interval in the left subtree can possibly extend far enough to overlap with the query[11]. This structural "foresight" allows the algorithm to aggressively prune massive, irrelevant branches of the tree without inspecting them individually[18].

Consequently, the Interval Tree identifies all overlapping bookings in $O(\log N + K)$ time, where K is the number of overlaps discovered[11]. This data structure is paramount for the rescheduling feature; when a booking is modified, the system removes its old interval from the tree in $O(\log N)$ time, queries the new interval, and if no overlaps exist, inserts the new interval, maintaining high performance during heavy change management[11].

The Sweep-Line Algorithm: Temporal Resource Aggregation

While the Interval Tree elegantly determines if a specific physical hall is free, verifying shared operational resources introduces a significantly more complex dimension[1]. Multiple non-overlapping halls at the same venue draw from the same limited pool of projectors, audio systems, and support staff concurrently[1]. A temporal query cannot simply ask "is a projector available?"; it must ask "at any microsecond during this 4-hour reservation, does the total aggregate number of projectors currently in use across all ongoing events exceed the venue's maximum inventory?"[1]. To resolve this continuous concurrency problem, the system employs a 1-Dimensional Sweep Line algorithm[19].

The Sweep Line algorithm calculates the peak concurrent usage of a resource over a defined timeline[21]. The implementation within the EventSync backend operates through the following sequence:

1. The system identifies the target venue and the new request's expanded temporal footprint. It queries the database for all confirmed bookings at that venue that intersect with this footprint[1].
2. For every relevant booking, the algorithm extracts two distinct "events" for each requested resource category: a StartEvent containing the timestamp and a positive quantity ($+Q$), and an EndEvent containing the timestamp and a negative quantity ($-Q$)[20].
3. The resource requirements of the new unconfirmed request are also transformed into start and end events and added to this collection.
4. All extracted events are sorted chronologically by their timestamp[20]. This is the most critical phase. If a StartEvent and an EndEvent from two different bookings share the exact same timestamp (e.g., one event finishes its cleanup buffer exactly at 12:00 PM, and another begins its setup buffer at 12:00 PM), the sorting comparator must be programmed to process the EndEvent first[12]. This ensures adherence to the half-open interval logic, recognizing that resources from a departing event instantly become available for an arriving event without causing a momentary, false spike in concurrent usage[12].
5. A running concurrency counter is initialized at zero. The algorithm sweeps sequentially through the sorted events, adding the positive and negative quantities to the running total[20].
6. If the running counter ever exceeds the venue's total capacity for that specific resource (e.g., the counter reaches 6 for Venue A staff, where the limit is 5), the resource constraint is actively violated[1]. The algorithm immediately halts, and the current hall candidate is rejected due to resource exhaustion[1].

The Sweep Line algorithm processes this aggregate resource verification in $O(N \log N)$ time, a complexity bound dominated entirely by the chronological sorting phase[20].

Comparative Analysis: Interval Trees vs. Sweep Line

The decision to utilize both an Interval Tree and a Sweep Line algorithm within the same architecture, rather than selecting one to handle all temporal logic, is driven by the distinct

computational strengths of each[24]. Interval Trees excel at dynamic querying and point-stabbing operations; they are optimized for environments with frequent insertions and deletions, making them ideal for the rapid spatial verification of individual halls as bookings are created, cancelled, or rescheduled[13].

Conversely, the Sweep Line algorithm is unmatched for batch intersection counting and determining maximum concurrency across a broad dataset[23]. While an Interval Tree could theoretically be queried repeatedly to find overlaps and then sum the resources of those overlaps, this approach becomes inefficient when managing aggregate pools across dozens of simultaneous events[23]. The Sweep Line processes all events chronologically, providing a definitive peak usage metric in a single pass[23]. By combining the Interval Tree for rapid spatial filtering and the Sweep Line for aggregate resource verification, the system achieves mathematically optimal, highly scalable constraint resolution[23].

Detailed Resolution of the Sample Situation

To definitively demonstrate the efficacy and mathematical correctness of these combined algorithms, the system must accurately resolve the exact "Sample Situation" provided in the hackathon problem constraints[1]. The algorithmic state transitions for this scenario are detailed below.

Initial System State: Venue A consists of Hall A1 (capacity 60) and Hall A2 (capacity 150). The localized shared resource pool contains a maximum of 2 projectors, 2 audio systems, and 5 staff members. The strictly enforced operating hours are 8:00 AM to 10:00 PM[1]. Both halls are currently empty.

Processing Request 1: The Conference.

- **Input Parameters:** 120 expected guests, requested event time of 10:00 AM to 1:00 PM, requiring 1 projector and 3 staff members[1].
- **Spatial Allocation:** The system filters for halls at Venue A with a capacity $\geq$ 120. Hall A2 (cap 150) is selected as the smallest viable candidate[1].
- **Temporal Expansion:** The 30-minute operational buffers are applied, resulting in a continuous reserved interval of 9:30 AM to 1:30 PM[1]. A boundary check confirms that [09:30, 13:30) falls entirely within the [08:00, 22:00) operating window.
- **Resource Verification:** The Sweep Line evaluates peak usage. Max concurrent projectors = 1 (limit 2). Max concurrent staff = 3 (limit 5).
- **Resolution:** All constraints are satisfied. The Conference is **Confirmed** in Hall A2. The Interval Tree for Hall A2 is updated with the node [09:30, 13:30).

Processing Request 2: The Workshop.

- **Input Parameters:** 50 expected guests, requested event time of 11:00 AM to 2:00 PM, requiring 1 projector and 2 staff members[1].
- **Spatial Allocation:** The system filters for halls at Venue A with a capacity $\geq$ 50. The sorted candidate list places Hall A1 (cap 60) first, followed by Hall A2[1].
- **Temporal Expansion:** Buffers are applied, creating a reserved interval of 10:30 AM to 2:30 PM, which passes the operating hours check[1].

- **Spatial Verification:** The Interval Tree for Hall A1 is queried. It is entirely empty. The spatial check passes immediately.
- **Resource Verification:** The Sweep Line is executed for Venue A. It aggregates the events for both the existing Conference and the proposed Workshop. Between 10:30 AM and 1:30 PM, both events overlap.
  - Projector concurrency: Conference (1) + Workshop (1) = 2. The peak usage equals the venue limit of 2, which is valid.
  - Staff concurrency: Conference (3) + Workshop (2) = 5. The peak usage equals the venue limit of 5, which is valid.
- **Resolution:** Both Request 1 and Request 2 are successfully **Confirmed** concurrently, operating in different halls but drawing from the same resource pool seamlessly[1].

Processing Request 3: The Product Launch.

- **Input Parameters:** 100 expected guests, requested event start time of 1:15 PM[1].
- **Spatial Allocation:** The system requires a hall with capacity $\geq$ 100. Hall A2 (cap 150) is the only viable candidate at Venue A[1].
- **Temporal Expansion:** A 1:15 PM event start time mathematically forces a reserved setup start time of 12:45 PM to satisfy the 30-minute buffer rule[1].
- **Spatial Verification Failure:** The system queries the Interval Tree for Hall A2 with the interval $[12:45\ \text{PM}, \text{End})$. The tree traverse instantly detects a fatal intersection with the existing Conference node $[09:30\ \text{AM}, 13:30\ \text{PM})$. The hall is physically occupied during the necessary setup window[1].
- **Resource Verification Failure:** Even if an arbitrary third hall of appropriate size magically existed at Venue A, the Sweep Line algorithm would independently reject the request. A 12:45 PM setup start overlaps with both the ongoing Conference and the Workshop. The staff required would be $3\ (\text{Conference}) + 2\ (\text{Workshop}) + X\ (\text{Launch}) > 5$. Any request requiring staff at 12:45 PM will cause immediate resource exhaustion[1].
- **Resolution:** The algorithm definitively rejects Venue A. Driven by the primary directives, the system autonomously traverses to Venue B to evaluate Hall B1 (capacity 100) or B2 (capacity 200) against the exact same temporal and resource constraints[1].

Core Functional Workflows and Change Management

The standard evaluation criteria dedicate a combined 45 marks to "Problem Understanding and Requirements Analysis" and "Technical Implementation and Code Quality"[1]. To secure these marks, the system must seamlessly execute complex state mutations without risking data corruption.

Robust Booking Change Management

The problem statement dictates that client cancellations must instantly release allocated resources, and any rescheduling attempts must completely re-evaluate every spatial and resource constraint[1]. Most critically, if a rescheduling attempt fails due to new conflicts, the

original confirmed booking must remain intact and unchanged[1].
This mandate necessitates the use of strict ACID-compliant database transactions (BEGIN, COMMIT, ROLLBACK) managed by the Node.js backend. The workflow for a reschedule request is explicitly defined:

1. The backend initiates a secure database transaction.
2. The system temporarily mutates the target booking's status from 'Confirmed' to a transitional 'Rescheduling' state. This is a vital architectural step; it exempts the original booking from the PostgreSQL EXCLUDE USING gist constraint checks and removes its resources from the Sweep Line aggregation, effectively freeing the space and equipment temporarily[8].
3. The algorithms execute the Interval Tree and Sweep Line validations against the newly requested time parameters and venue limits[1].
4. If all constraints pass, the transaction executes an UPDATE with the new times, reverts the status to 'Confirmed', and commits the transaction to disk[1].
5. If any constraint fails, the backend intercepts the failure and immediately executes a ROLLBACK command. This instantly aborts all transitional changes, restoring the database to its precise state prior to step 1, thereby preserving the original confirmed booking flawlessly as required by the rubric[1].

Decision Visibility and Granular Audit Logging

The application must present "an understandable reason whenever a request cannot be accommodated"[1]. Returning a generic HTTP 400 "Booking Failed" response represents a severe failure in user experience and technical implementation. The backend algorithmic engine must be capable of generating highly specific diagnostic payloads.
For example, when Request 3 (the Product Launch) fails at Venue A, the JSON payload returned to the frontend should precisely enumerate the point of failure: {"status": "failed", "reason": "Resource Exhaustion", "venue": "Venue A", "details": "Venue A possesses a maximum limit of 5 support staff. Your setup interval beginning at 12:45 PM requires staff allocation, but 5 staff members are already fully allocated to concurrent confirmed events (Conference_ID:12, Workshop_ID:45) between 12:45 PM and 1:30 PM."}. This level of granular, transparent feedback demonstrates a masterful technical implementation to the judging panel and mirrors the diagnostic capabilities of premium enterprise venue software[2].

High-Value Innovative Features for the Hackathon

The hackathon explicitly encourages participating teams to implement "at least one additional, useful feature of its own choice," with the caveat that teams must thoroughly explain the real-world need it addresses and smoothly demonstrate its implementation[1]. Relevant, well-executed enhancements distinguish stronger solutions and heavily influence the "Algorithm Design" and "Project Demonstration" scoring categories[2]. The EventSync system should incorporate a synergistic combination of the following advanced features, drawing directly from the capabilities of industry-leading venue management platforms[3].

Alternative Time Slot Recommendation Engine

The Real-World Need: In traditional venue management paradigms, when a client requests a specific time slot that is unavailable, the interaction abruptly terminates with a rejection. The client is forced into a frustrating cycle of blindly guessing alternative times or engaging in lengthy phone consultations with the venue manager[30]. A modern, intelligent system should emulate the service of a human concierge by proactively analyzing the venue's schedule and suggesting the closest available alternatives, thereby reducing friction and capturing revenue that would otherwise be lost to competitors[31].

The Implementation: If the primary allocation algorithm fails due to spatial overlaps or resource exhaustion, the backend does not simply return an error; it triggers an asynchronous "Recommendation Module" utilizing a First-Insert Search algorithm[33].

- **Time-Shifting:** The algorithm recursively iterates forward and backward from the client's originally requested time, typically in 30-minute operational increments[33]. At each step, it executes the Interval Tree and Sweep Line checks until it successfully locates the nearest valid, unconstrained block of time at the preferred venue that can accommodate both the spatial and resource requirements[33].
- **Venue-Shifting:** Concurrently, the algorithm evaluates the exact original time request against the remaining venues in the enterprise portfolio (e.g., if Venue A failed, it checks the availability of Halls B1, B2, C1, and C2)[30].
- The frontend intercepts this augmented diagnostic payload and presents the options visually to the user: *"Your requested time of 1:15 PM is unavailable at Venue A due to staff limitations. However, you can book Hall A2 starting at 2:00 PM today, or you may secure Hall B2 at your originally requested time of 1:15 PM."*[31]. This proactive problem-solving drastically improves client satisfaction and demonstrates advanced algorithmic traversal to the judges[31].

Automated Waitlist and Cascade Booking Algorithms

The Real-World Need: Client cancellations represent a direct loss of projected revenue. Manually tracking prospective clients who expressed interest in a specific, already-booked time slot, and subsequently contacting them when that slot suddenly opens up, is an administratively heavy and unacceptably slow process[37]. By the time staff reaches out, the prospective client has often booked elsewhere, resulting in the venue slot remaining permanently empty[38].

The Implementation: The architecture introduces a highly optimized Waitlist relational table. When a client requests a highly desirable but currently occupied slot, the frontend allows them to opt-in to an automated waitlist[37].

- When an existing confirmed booking is cancelled, the system triggers a background asynchronous job—the Cascade Algorithm—prior to returning the HTTP response to the cancelling user[37].
- The Cascade Algorithm queries the database for all waitlisted requests whose temporal parameters overlap with the newly freed interval.
- It sorts these waitlisted requests by priority or chronological order of entry, and sequentially runs the standard hall and Sweep Line resource allocation checks against

them[37].

- If a waitlisted request successfully passes all constraints, the system automatically executes a transaction to convert the waitlist entry into a Confirmed booking, simultaneously dispatching an automated notification (which can be simulated via UI alerts or console logs for the hackathon demonstration)[37].
- This feature demonstrates mastery over asynchronous event-driven architecture and directly addresses a critical operational metric: maximizing venue occupancy rates and profitability[3].

Interactive Resource Utilization Timeline (Gantt View)

The Real-World Need: Venue managers cannot efficiently operate a complex, multi-hall facility by reading raw text rows in a database. They require a holistic, highly visual overview of the day's operations to instantly spot scheduling bottlenecks, manage staff shift changes, and anticipate chaotic setup and teardown periods[3].

The Implementation: The frontend architecture will integrate the robust FullCalendar library for React, specifically licensing the premium @fullcalendar/resource-timeline plugin, which perfectly maps to the physical reality of venue management[40].

- The Y-axis of the timeline hierarchically displays the Venues, and nested beneath them, their respective Halls[4].
- The X-axis represents the strictly enforced 8:00 AM to 10:00 PM operational timeline.
- Confirmed bookings are rendered as interactive blocks on the timeline. A crucial visual enhancement, demonstrating deep domain understanding, involves color-coding the core event time (e.g., solid blue) and rendering the mandatory 30-minute setup and cleanup buffers as distinct, hashed, or semi-transparent extensions of the main block[40]. This allows managers to visualize exactly when staff is required versus when the event is simply occurring.
- Clicking an event block invokes a contextual side panel detailing the precise allocation of projectors, audio systems, and staff for that specific interval[42]. This Gantt visualization provides unparalleled "Decision Visibility," allowing the judges to instantly comprehend the system's state during the project demonstration[1].

Digital Banquet Event Orders (BEOs) and Offline Mobile Functionality

The Real-World Need: In live venue operations, the "Banquet Event Order" (BEO) is the master document detailing every requirement for an event. Historically, these are printed packets distributed to audiovisual, housekeeping, and security teams[29]. When last-minute headcount or equipment revisions occur, tracking down physical packets to update them is impossible, leading to operational failures[29]. Furthermore, operational staff frequently work in concrete basements, historic stone buildings, or convention center subfloors where Wi-Fi connectivity is non-existent, rendering cloud-only web applications useless during critical moments[29].

The Implementation: The EventSync application features a dedicated "Staff Portal" view optimized for mobile devices.

- The system generates living digital BEOs. If a client reschedules or alters their projector requirements, the change propagates instantly via WebSockets to all active staff

devices[29].

- To address the connectivity issue, the frontend utilizes Service Workers and IndexedDB to cache critical daily schedule data locally on the device[29].
- Staff can continue viewing their assigned schedules, capturing notes, and marking setup tasks as complete even when deep in a dead zone without a signal[29].
- The application queues these state changes locally. When the device regains connectivity, the application intelligently synchronizes the queued changes with the PostgreSQL backend, utilizing timestamped audit trails for quality assurance and dispute resolution[29]. This level of operational realism elevates the project from a simple hackathon toy to a viable commercial prototype.

Predictive Preventative Maintenance Workflows

The Real-World Need: In high-traffic, multi-venue environments, shared technical equipment—particularly high-wattage projectors and complex audio systems—degrades with continuous use. If a venue relies on reactive "break-fix" cycles, a projector is likely to fail mid-event, destroying client trust and potentially incurring liability[43]. Shifting to scheduled, preventative maintenance prevents catastrophic failures during peak seasons[43].

The Implementation: The database schema is expanded to track individual assets (e.g., Projector_001, Projector_002) rather than relying solely on raw aggregate counts[43].

- The system actively tracks the cumulative runtime hours each specific piece of equipment is booked[43].
- Once an asset surpasses a predefined utilization threshold (e.g., 50 hours of active booking time), the algorithmic engine autonomously generates a specialized "Maintenance Block" interval in the database for that asset.
- This maintenance block effectively removes the asset from the available Sweep Line pool for a predefined 2-hour window, strategically scheduled during historical off-peak hours to minimize disruption[43].
- This feature demonstrates a profound understanding of total lifecycle asset tracking and operational risk management, highly sought-after capabilities in enterprise facility management software[43].

Alignment with the Official Evaluation Rubric

To achieve a perfect score of 100, the development roadmap and the final project presentation must align meticulously with the hackathon's official evaluation rubric[1].

1. Problem Understanding and Requirements Analysis (15 Marks) The foundation of a high score here is absolute adherence to the stated constraints. The UI must aggressively restrict clients from requesting times outside the 8:00 AM–10:00 PM operating hours, and the backend must flawlessly append and calculate the 30-minute buffers without violating the daily boundaries[1]. Furthermore, the initial dataset must be modeled dynamically in the database, proving that the system can accommodate the addition of a hypothetical "Venue D" or a new resource type like "Lighting Rigs" without requiring underlying code modifications[1].

2. Algorithm Design and Problem-Solving (20 Marks) This category is where the

implementation of the Interval Tree and Sweep Line algorithms will secure maximum points. During the technical code review, the presentation should explicitly explain how the Sweep Line algorithm turns a computationally disastrous $O(N^2)$ brute-force intersection check into an elegant, scalable $O(N \log N)$ chronological sweep[20]. The team must confidently present the half-open interval logic to prove that mathematical edge cases—specifically back-to-back bookings sharing a boundary minute—are handled flawlessly without off-by-one errors[7].

3. Technical Implementation and Code Quality (30 Marks) Earning 30 marks requires a modern, cleanly decoupled stack (React frontend, Node.js backend, PostgreSQL database) utilizing standard RESTful API or GraphQL paradigms. The defining technical achievement will be the implementation of the PostgreSQL EXCLUDE USING gist constraint[8]. Explaining this implementation proves a deep understanding of database-level concurrency control, an enterprise-grade technique that immediately sets the solution apart from competing teams relying on basic, vulnerable SQLite or MySQL WHERE clauses[5]. The codebase must feature modular architecture, descriptive variable nomenclature, and comprehensive error handling that catches database integrity faults and surfaces them smoothly to the UI.

4. Project Demonstration (15 Marks) The live demonstration must be heavily rehearsed to visually prove that the complex constraints operate in real-time. The team should recreate the exact "Sample Situation" from the prompt live on screen.

1. Book the 120-person Conference.
2. Book the 50-person Workshop. Show the UI confirming both successfully at Venue A.
3. Attempt to force the 100-person Product Launch at 1:15 PM into Venue A. When the backend predictably rejects the third request, the UI should instantly display the "Alternative Time Slot Recommendation" engine in action, suggesting Venue B[31]. Subsequently, the team should navigate to the FullCalendar Gantt view to visually prove *why* the failure occurred, pointing to the overlapping operational blocks and the resource exhaustion alerts on the timeline[40].

5. Responses to Judges' Questions (20 Marks) The team must proactively anticipate aggressive questioning regarding system scalability and race conditions. If a judge poses the scenario, "What happens if 1,000 different clients attempt to book Hall A1 for the exact same Saturday time slot at the exact same millisecond?", the response must confidently detail the database-level isolation. The team will explain that the application logic does not handle the lock; rather, the PostgreSQL exclusion constraints ensure that the first transaction to acquire the write lock succeeds, while the remaining 999 transactions are safely rejected by the database engine, rolled back by the server, and gracefully presented with alternative time slots via the recommendation engine, ensuring zero data corruption[8].

Conclusion

The EventSync problem statement is a sophisticated resource-constrained scheduling challenge cleverly disguised as a standard web development task. Competing teams that rely on naive algorithmic approaches—such as rudimentary nested for-loops for time checking and flat SQL queries—will rapidly encounter catastrophic edge cases, unmanageable race conditions, and severe computational bottlenecks during testing.

By architecting the EventSync solution around mathematically optimal data structures—specifically leveraging Interval Trees for instantaneous spatial availability filtering and Sweep Line processing for concurrent resource aggregation—the system ensures absolute correctness and high-performance scalability. Furthermore, enforcing these temporal rules at the deepest structural level using PostgreSQL exclusion constraints guarantees zero double-bookings, regardless of the severity of server concurrency or network latency. By overlaying these impenetrable backend technologies with an intuitive, highly visual React-based Gantt interface and intelligent, proactive features like alternative time-slot recommendations and automated waitlist cascading, the resulting application successfully transitions from a basic hackathon prototype to an enterprise-grade commercial product, perfectly positioned to maximize every criterion in the evaluation rubric.

Works cited

1. EventSync_Hackathon_Problem_Statement.pdf
2.
3. Your Venue Management Guide - Momentus Technologies, https://gomomentus.com/a-guide-to-venue-management-software
4. Venue Management Software Guide for Operators - Skedda, https://www.skedda.com/insights/venue-management-software
5. frankensqlite/README.md at main - GitHub, https://github.com/Dicklesworthstone/frankensqlite/blob/main/README.md
6. Postgres vs MySQL vs SQLite: Which One to Use - Chat2DB, https://chat2db.ai/resources/blog/postgres-vs-mysql-vs-sqlite
7. SQL BETWEEN & Range Queries: Numeric, Date, Inclusive vs, https://pipecode.ai/blogs/sql-between-range-queries-numeric-date-inclusive-vs-exclusive
8. PostgreSQL Constraints: column and table | Prisma's Data Guide, https://www.prisma.io/dataguide/postgresql/column-and-table-constraints
9. Free Room reservations overlapping intervals practice dataset, https://databasesample.com/datasets/room-reservations-overlapping-intervals-practice-dataset
10. Top 10 Event Venue Management Tools: Features, Pros, Cons, https://www.myhospitalnow.com/blog/top-10-event-venue-management-tools-features-pros-cons-comparison/
11. Interval Trees & Overlap Detection — Learning Notebook, https://www.javalld.com/learn/interval-tree
12. Meeting Rooms: Solve It Online in 15 Languages - Coddy Tech, https://coddy.tech/problems/meeting-rooms
13. Algorithm to detect overlapping periods - Codemia, https://codemia.io/knowledge-hub/path/algorithm_to_detect_overlapping_periods_duplicate_1
14. How to Prevent Double Booking: Locks, Holds and Expiry, https://dev.to/muhammadalimughal/how-to-prevent-double-booking-locks-holds-and-expiry-5e54

15. sqlite - Alternative to using subquery inside CHECK constraint?, https://stackoverflow.com/questions/42171195/alternative-to-using-subquery-inside-check-constraint
16. A Parallel Data Distribution Management Algorithm 1 1 ... - arXiv, https://arxiv.org/html/1309.3458v3
17. DSA Module 2: Interval Trees Overview | PDF - Scribd, https://www.scribd.com/document/878433475/PG-DSA-MOD-2-NOTES
18. Interval Tree | Bohrium, https://www.bohrium.com/en/sciencepedia/feynman/keyword/interval_tree
19. Sweep-Line Algorithm for Spiral Pocketing | PDF - Scribd, https://www.scribd.com/document/202607704/12-148-1-PB-1-pdf
20. Sweep Line Algorithm - Implementation for 1D plane - Stack Overflow, https://stackoverflow.com/questions/32263145/sweep-line-algorithm-implementation-for-1d-plane
21. Mastering Overlapping Interval Problems in Competitive Programming, https://www.rishabhxchoudhary.com/blog/overlapping-interval-problems
22. LeetCode 253 Meeting Rooms II Solution & Explanation | NeetCode, https://neetcode.io/solutions/meeting-rooms-ii
23. Leetcode 253. Meeting Rooms II | Hello Interview, https://www.hellointerview.com/community/questions/meeting-rooms-2/cm5eh7nrh04u1838orn6jicpz
24. A fast adaptive algorithm for computing whole-genome homology, https://pmc.ncbi.nlm.nih.gov/articles/PMC6129286/
25. The Algorithm Atlas - Assets - Chirag Patnaik, https://assets.chiragpatnaik.com/algorithm-atlas.html
26. (PDF) In-Memory Interval Joins - ResearchGate, https://www.researchgate.net/publication/350736571_In-Memory_Interval_Joins
27. Mastering Intervals: From LeetCode Challenges to Real-World, https://medium.com/@hanxuyang0826/mastering-intervals-from-leetcode-challenges-to-real-world-applications-1b72eeef1b98
28. Algorithms & Data Structures - Senior Stack, https://senior-stack.uz/Interview%20Question/04-algorithms-and-data-structures/
29. Venue Management Software with Mobile App: 2026 Guide, https://nexteventapp.com/venue-management-software-with-mobile-app/
30. Multi-agent Pareto Appointment Exchanging in Hospital Patient, https://ivanvermeulen.nl/soca.pdf
31. Discover the latest features boosting your calendar experience - Zoho, https://www.zoho.com/blog/calendar/boost-your-calendar-experience.html
32. Integrating AI voice tools with Google Calendar - ElevenLabs, https://elevenlabs.io/blog/integrating-ai-voice-tools-with-google-calendar
33. Managing Concurrent Interactions in Online Time Slot Booking, https://pubsonline.informs.org/doi/10.1287/trsc.2022.0445
34. A Multi-Task Dynamic Scheduling Method for Space Launch TT&C, https://www.mdpi.com/2226-4310/13/7/605

35. Facility Reservation System with Priority-Based Internal-External, https://www.researchgate.net/publication/416434063_Facility_Reservation_System_with_Priority-Based_Internal-External_Conflict_Resolution
36. Parent-Teacher Conference Time Slot Selection Form Template, https://www.jotform.com/form-templates/parent-teacher-conference-time-slot-selection
37. How to Fill Canceled Appointments With an Automated Waitlist, https://www.picktime.com/resources/how-to-fill-canceled-appointments-with-an-automated-waitlist/
38. Automated Waitlist Management: How to Fill Appointment ... - Emitrr, https://emitrr.com/blog/automated-waitlist-management/
39. Best resource management software 2026 - Dayshape, https://dayshape.com/insights/what-are-the-top-10-resource-management-tools-for-firms-with-500-employees
40. Demos - FullCalendar, https://fullcalendar.io/demos
41. @fullcalendar/react | Yarn, https://classic.yarnpkg.com/en/package/@fullcalendar/react
42. React Component - Docs v6 - FullCalendar, https://legacy.fullcalendar.io/v6/react
43. Venue Management Software: Top 8 Features to Look For, https://www.247software.com/blog/venue-management-software-top-8-features-to-look-for
44. A Comprehensive Guide to Venue Management - Mitti, https://mitti.com/topics/venue-management